

RAG Interpretativo y Compresión de modelos: Sistema de clasificación de lyrics en géneros musicales explicable y eficiente

Procesamiento del Lenguaje Natural - Práctica 2

Diego Esclarín Fernández
Daniel del Nuevo Montero
Carmen Maeso Soler

Grado en Inteligencia Artificial

Curso 2025-2026



Universidad
Rey Juan Carlos

Escuela Técnica Superior
Ingeniería Informática

Índice

1. Introducción	2
2. Módulo de Retrieval Denso y Clasificador k-NN	3
2.1. Generación de Embeddings e Índice k-NN	3
2.2. Evaluación del Módulo de Recuperación	3
2.3. Clasificador k-NN	3
3. Clasificador Híbrido (Transformer + k-NN)	4
3.1. Metodología	4
3.2. Resultados	4
3.3. Análisis	4
4. Modelo Student y Eficiencia	5
4.1. Entrenamiento y Evaluación	5
4.2. Análisis de Eficiencia	5
5. Proyecto de Python: Resumen Automático y Herramienta Interactiva	6
5.1. Resumen Global por Clase	6
5.2. Explicabilidad de Predicciones en el Notebook	6
6. Análisis de Resultados y Discusión Final	7
6.1. Comparativa de Modelos: Calidad vs. Coste vs. Interpretabilidad	7
6.2. Análisis de Equidad y Sesgo	8
6.3. Reflexión sobre la Explicabilidad y Riesgos	9
7. Uso de Gemini	10

1. Introducción

El objetivo de este proyecto es avanzar en la clasificación automática de letras de canciones, construyendo sobre los resultados obtenidos en la práctica anterior. En la primera fase, se exploraron modelos clásicos basados en frecuencia de términos (TF-IDF con regresión logística) y modelos de aprendizaje profundo basados en Transformers (*Fine-tuning* de RoBERTa).

En esta segunda etapa, nos centramos en mejorar y analizar estos sistemas mediante técnicas más avanzadas y eficientes, abordando tres áreas principales:

1. **Recuperación Densa (Dense Retrieval) y Clasificación k-NN:** Utilizamos los vectores de características (*embeddings*) generados por el modelo Transformer para realizar búsquedas de similitud semántica. Esto nos permite clasificar nuevas canciones basándonos en sus vecinos más cercanos en el espacio vectorial, un enfoque no paramétrico que ofrece interpretabilidad y flexibilidad.
2. **Modelos Híbridos:** Combinamos la predicción del modelo Transformer (Teacher) con la información del clasificador k-NN. Este enfoque híbrido busca aprovechar lo mejor de ambos mundos: la capacidad de generalización del modelo profundo y la evidencia local aportada por los ejemplos de entrenamiento más similares.
3. **Destilación de Conocimiento (Knowledge Distillation) y Eficiencia:** Dada la alta demanda computacional de los modelos Transformers grandes, implementamos una técnica de destilación para entrenar un modelo más pequeño (“Student”, DistilRoBERTa) que imite el comportamiento del modelo grande (“Teacher”). El objetivo es mantener una alta precisión reduciendo significativamente el tamaño del modelo y el tiempo de inferencia.

Finalmente, se pone un énfasis especial en la **explicabilidad** y el análisis de errores, utilizando técnicas de recuperación para mostrar ejemplos similares y generando resúmenes automáticos para caracterizar cada género musical (Rap, Rock, Pop, Other).

Es importante destacar que todos los experimentos se han realizado sobre un conjunto de datos particionado en entrenamiento, validación y test. Los resultados presentados en esta memoria provienen de la ejecución del cuaderno `notebook.ipynb`, utilizando los modelos previamente entrenados y evaluados sobre el conjunto de test completo.

2. Módulo de Retrieval Denso y Clasificador k-NN

La primera fase del proyecto ha consistido en la implementación de un sistema de recuperación denso (*Dense Retrieval*). A diferencia de los métodos clásicos como TF-IDF que buscan coincidencias exactas de palabras (representaciones dispersas), el retrieval denso utiliza vectores numéricos densos (*embeddings*) que capturan el significado semántico del texto.

2.1. Generación de Embeddings e Índice k-NN

Para representar las letras de las canciones, hemos utilizado el codificador (*encoder*) del modelo Transformer “Teacher” (RoBERTa-base), entrenado en la práctica anterior. Para cada canción, se extrae el vector correspondiente al token especial [CLS] de la última capa oculta, obteniendo una representación vectorial de alta dimensionalidad.

Con estos embeddings, se ha construido un índice de búsqueda utilizando el algoritmo de k-vecinos más cercanos (k-NN) con la métrica de similitud del coseno. Se ha configurado el sistema para recuperar los $k = 5$ vecinos más similares para cada consulta.

2.2. Evaluación del Módulo de Recuperación

Para evaluar la calidad de los vecinos recuperados, se han calculado métricas de precisión y cobertura (*recall*) sobre el conjunto de test:

- **Precision@5:** 0.7834. Esto indica que, en promedio, casi 4 de los 5 vecinos recuperados pertenecen al mismo género musical que la canción de consulta, lo cual es un resultado muy positivo y sugiere que los embeddings agrupan correctamente las canciones por género.
- **Recall@5:** 0.0001. El valor medio obtenido es bajo debido a la naturaleza de la métrica en conjuntos de datos grandes (donde hay miles de ejemplos relevantes y solo recuperamos 5), por lo que la precisión es una métrica más informativa en este contexto de “búsqueda de los más similares”.

2.3. Clasificador k-NN

Aprovechando el índice construido, hemos implementado un clasificador k-NN que predice la etiqueta de una canción mediante una votación mayoritaria de las etiquetas de sus 5 vecinos más cercanos.

Los resultados sobre el conjunto de test han sido excelentes:

- **Accuracy:** 0.806
- **Macro-F1:** 0.806

Si comparamos estos resultados con el modelo **Baseline** (TF-IDF + Clasificador Lineal) desarrollado en la práctica 1, que obtenía una accuracy de **0.648**, observamos una mejora sustancial de más de 15 puntos porcentuales. Esto confirma que la representación semántica aprendida por el Transformer es mucho más rica y discriminativa que la simple frecuencia de palabras. Además, el clasificador k-NN alcanza un rendimiento muy cercano al del propio modelo Teacher (0.803), pero con la ventaja de ser un modelo no paramétrico que permite interpretar las decisiones observando los ejemplos vecinos.

3. Clasificador Híbrido (Transformer + k-NN)

Tras evaluar el modelo Transformer y el clasificador k-NN por separado, hemos implementado un **Clasificador Híbrido** inspirado en las técnicas de RAG (Retrieval-Augmented Generation) pero aplicado a clasificación. La idea fundamental es combinar la predicción paramétrica del modelo Transformer (“Teacher”) con la evidencia no paramétrica extraída de los vecinos más cercanos.

3.1. Metodología

El modelo híbrido integra las puntuaciones de ambos sistemas mediante un parámetro de interpolación α (alpha). De esta forma, la decisión final se toma ponderando la confianza del modelo profundo y la información local del retrieval. Hemos experimentado con distintos valores de $\alpha \in \{0, 0,25, 0,5, 0,75, 1,0\}$, donde:

- $\alpha = 0,0$ equivale a usar puramente el clasificador k-NN (basado en vecinos).
- $\alpha = 1,0$ equivale a usar puramente el modelo Transformer (Teacher).
- Valores intermedios representan una combinación de ambos.

3.2. Resultados

Al evaluar las distintas configuraciones sobre el conjunto de test, hemos obtenido los siguientes resultados de *Accuracy* y *Macro-F1*:

Alpha (α)	Accuracy	Macro-F1
0.0 (k-NN puro)	0.803	0.804
0.25	0.806	0.807
0.50	0.808	0.809
0.75	0.807	0.808
1.0 (Teacher puro)	0.806	0.806

Cuadro 1: Resultados del clasificador híbrido según el valor de alpha.

3.3. Análisis

El mejor rendimiento se obtiene con $\alpha = 0,5$, alcanzando una **Accuracy de 0.808**. Aunque la mejora respecto a los modelos individuales (0.803 y 0.806) es modesta, el experimento demuestra que ambos sistemas aportan información ligeramente complementaria. El modelo Transformer captura patrones globales complejos, mientras que el k-NN refuerza la decisión basándose en ejemplos locales muy similares, ayudando a corregir errores en casos frontera. Sin embargo, dado que ambos modelos se basan en la misma representación vectorial (los embeddings del Teacher), es esperable que sus predicciones estén altamente correlacionadas, limitando el margen de mejora del enfoque híbrido.

4. Modelo Student y Eficiencia

Uno de los desafíos de los modelos basados en Transformers como RoBERTa es su alto coste computacional y tamaño en memoria. Para abordar esto, hemos explorado el uso de un modelo “Student” basado en **DistilRoBERTa**, una versión destilada, más pequeña y rápida de RoBERTa.

4.1. Entrenamiento y Evaluación

El modelo Student ha sido sometido a un proceso de fine-tuning utilizando el mismo conjunto de datos que el modelo Teacher. Tras 3 épocas de entrenamiento, se ha evaluado su rendimiento en el conjunto de test:

- **Accuracy:** 0.796
- **Macro-F1:** 0.797

Comparado con el modelo Teacher (Acc 0.803), observamos una caída de rendimiento mínima (menos de 1 punto porcentual). Esto indica que el modelo comprimido es capaz de retener la gran mayoría del conocimiento necesario para la tarea de clasificación de géneros.

4.2. Análisis de Eficiencia

La principal ventaja del modelo Student reside en su eficiencia. Hemos medido el tiempo de inferencia sobre el conjunto completo de test (47.638 ejemplos) y el tamaño en disco del modelo:

Modelo	Accuracy	Tiempo Inferencia (s)	Velocidad Relativa
Teacher (RoBERTa)	0.803	93.86	1.0x
Student (DistilRoBERTa)	0.796	42.19	~2.2x

Cuadro 2: Comparación de eficiencia y rendimiento entre Teacher y Student.

Conclusión: El modelo Student es más del doble de rápido que el Teacher y su tamaño en disco es significativamente menor (313 MB), con una pérdida de calidad casi imperceptible. Esto lo convierte en la opción ideal para despliegues en entornos de producción o dispositivos con recursos limitados.

5. Proyecto de Python: Resumen Automático y Herramienta Interactiva

Como complemento a las métricas cuantitativas, se ha explorado el uso de modelos generativos de resumen (*Summarization*) para mejorar la interpretabilidad del sistema y ofrecer explicaciones en lenguaje natural. Esta funcionalidad se ha integrado y demostrado directamente en el cuaderno de experimentos (`notebook.ipynb`).

5.1. Resumen Global por Clase

En una primera aproximación explicativa, se buscó caracterizar automáticamente cada género musical. Para ello, se seleccionaron aleatoriamente 50 canciones de cada clase del conjunto de entrenamiento y se concatenaron sus letras para formar un “pseudo-documento” representativo. Posteriormente, se utilizó un modelo de resumen (basado en la arquitectura T5 o BART, implementado en la clase `Summarizer`) para generar una descripción concisa de 3 a 5 frases.

El objetivo de esta técnica es extraer los temas y patrones léxicos dominantes de cada género sin intervención humana. Los resultados obtenidos muestran cómo el modelo es capaz de capturar la “esencia” textual de las categorías, identificando jerga específica en el Rap o temáticas recurrentes en el Pop y el Rock.

5.2. Explicabilidad de Predicciones en el Notebook

En lugar de una aplicación externa, la lógica de explicabilidad se ha implementado como parte del flujo de análisis dentro del propio *notebook*. Esto permite inspeccionar dinámicamente el comportamiento de los modelos (especialmente el Híbrido y el Student) sobre ejemplos concretos del conjunto de test.

El flujo de explicabilidad implementado realiza las siguientes acciones para un ejemplo dado:

1. **Predicción y Contexto:** Se muestra el texto de la canción, su etiqueta real y la predicción del modelo.
2. **Explicación basada en Recuperación (Retrieval):** El sistema recupera los $k = 5$ vecinos más cercanos del conjunto de entrenamiento utilizando la similitud de embeddings. Esto proporciona una “evidencia” directa de por qué el modelo ha tomado una decisión, permitiendo al analista ver si la canción de test se agrupa semánticamente con canciones del mismo género o si, en caso de error, se parece más a ejemplos de otra clase.
3. **Análisis de Errores:** El código selecciona automáticamente ejemplos donde el modelo ha fallado y muestra sus vecinos. Esto es crucial para distinguir entre errores del modelo y ambigüedades en los datos (por ejemplo, una canción de Rock cuyos vecinos más cercanos son todos Pop, sugiriendo una frontera difusa entre géneros).

Esta integración en el cuaderno facilita una validación cualitativa inmediata de los modelos entrenados, complementando las métricas de Accuracy y F1 con una comprensión profunda de los aciertos y fallos del sistema.

6. Análisis de Resultados y Discusión Final

Por ultimo, comparamos todas las técnicas implementadas, analizando los compromisos entre eficiencia y calidad, y reflexionando sobre los aspectos éticos y de explicabilidad.

6.1. Comparativa de Modelos: Calidad vs. Coste vs. Interpretabilidad

En la Tabla 3 y la Figura 1 se resume el rendimiento de los modelos desarrollados frente al estado del arte (Teacher) y el baseline de la práctica anterior.

Modelo	Accuracy	Macro-F1
Hybrid ($\alpha = 0,5$)	0.808	0.809
k-NN (embeddings)	0.806	0.806
Teacher Transformer	0.803	0.804
Student (distilled)	0.796	0.797
Baseline (TF-IDF)	0.648	0.646

Cuadro 3: Resumen comparativo de rendimiento en el conjunto de test.

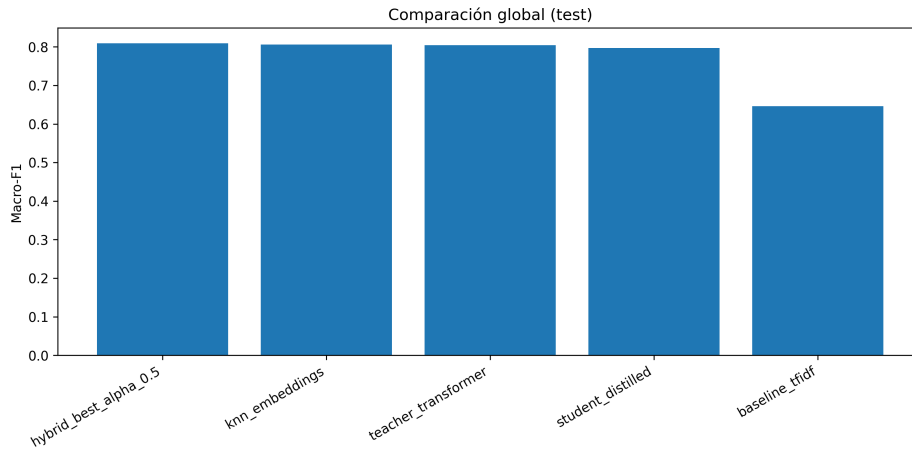


Figura 1: Comparativa de Macro-F1: Los modelos basados en Transformers superan ampliamente al Baseline.

Análisis del Equilibrio

1. **Calidad (Acc/F1):** Los modelos basados en representaciones densas (Teacher, k-NN, Híbrido, Student) son claramente superiores al Baseline TF-IDF, logrando una mejora del $\sim 24\%$ en Accuracy (de 0.65 a $> 0,80$). Entre ellos, el **Híbrido** ($\alpha = 0,5$) es marginalmente el mejor (0.809 Macro-F1), aunque la diferencia con el Teacher (0.804) y el k-NN Puro (0.806) es mínima.
2. **Eficiencia (Coste):**
 - El **Student (DistilRoBERTa)** es el claro ganador en eficiencia, siendo **2.2 veces más rápido** en inferencia que el Teacher y ocupando un 40 % menos de espacio en disco, con una pérdida de calidad despreciable (-0.007 en Accuracy).
 - El **k-NN**, aunque no requiere entrenamiento de una red neuronal (solo inferencia del encoder), tiene un coste computacional en tiempo de inferencia que crece con el tamaño del conjunto de datos (búsqueda de vecinos), lo cual puede ser prohibitivo en escalas masivas si no se usan índices aproximados (como FAISS).
3. **Interpretabilidad:**
 - Los modelos **Base (Teacher/Student)** actúan como cajas negras.
 - El **Baseline (Lineal)** ofrece explicaciones basadas en pesos de palabras (tokens), útiles pero limitadas al no capturar contexto.
 - El **k-NN y el Híbrido** ofrecen la mejor interpretabilidad semántica al proporcionar *explicaciones basadas en ejemplos* (“es de género X porque se parece a estas canciones...”), lo cual genera mayor confianza en el usuario no técnico.

Conclusión: La combinación que ofrece el mejor equilibrio global depende del caso de uso. Para despliegues en producción con recursos limitados, el **Modelo Student** es la elección óptima. Si la prioridad es la transparencia y la confianza del usuario, el sistema **Híbrido (o k-NN)** es preferible por su capacidad explicativa inherente.

6.2. Análisis de Equidad y Sesgo

El análisis de rendimiento por clase (Figura 2) muestra una disparidad notable en el Recall.

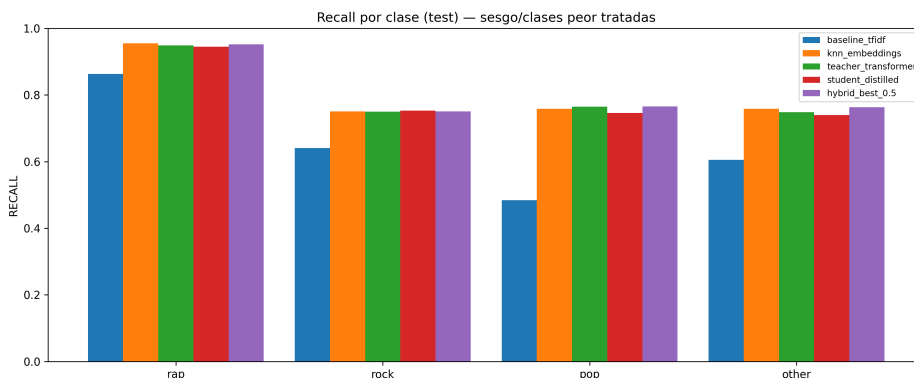


Figura 2: Recall por clase: Rap es fácilmente identificable; Rock y Pop presentan confusión.

- **Clases Desfavorecidas:** Las clases *Rock* y *Pop* tienen peores resultados (Recall ~ 0.70 - 0.75) en comparación con el *Rap* ($> 0,95$). Sin embargo, esto no parece indicar un sesgo de representación como tal, sino una **ambigüedad estructural**: estos géneros comparten mucho vocabulario y temáticas, haciendo que la frontera de decisión sea difusa incluso para humanos.
- **Riesgo de los Vecinos:** El uso de k-NN conlleva un riesgo específico: si el conjunto de entrenamiento contiene sesgos (por ejemplo, textos ofensivos agrupados consistentemente bajo una etiqueta), el sistema de recuperación **reforzará estos sesgos**. Si una canción nueva contiene términos ofensivos y recupera 5 vecinos ofensivos del género X, el modelo la clasificará como X con alta confianza, perpetuando estereotipos.

No obstante, es probable que parte de las diferencias observadas se deban a un **sesgo de muestreo en la construcción del conjunto de datos**. La limitada disponibilidad de ejemplos para ciertos géneros obligó a agruparlos en la categoría *other*, lo que da lugar a una clase altamente heterogénea y con fronteras semánticas poco definidas. Como consecuencia, los modelos no aprenden patrones consistentes asociados a esta categoría, sino que se ven forzados a discriminar entre un conjunto reducido de clases bien representadas, lo que penaliza especialmente el recall de las clases más diversas.

6.3. Reflexión sobre la Explicabilidad y Riesgos

La implementación de diferentes técnicas de explicabilidad nos permite comparar su utilidad real:

1. **Explicaciones por Tokens (Baseline P1)**: Son útiles para depurar el modelo (detectar *stopwords* con mucho peso o “atajos” espurios), pero fallan al explicar decisiones basadas en el tono, la ironía o el contexto global de la canción.
2. **Explicaciones por Ejemplos (Retrieval)**: Son extremadamente útiles para validar la calidad de los datos. Si un ejemplo de test recupera vecinos con etiquetas incorrectas en el conjunto de entrenamiento, permite detectar errores de etiquetado en la fuente (ruido en las etiquetas).
3. **Explicaciones Generativas (Resumen)**: Ayudan a comprender “de qué trata” una clase a alto nivel. Sin embargo, conllevan el **riesgo de alucinaciones**: el modelo podría generar descripciones plausibles pero desconectadas de los datos reales si no está bien condicionado, lo que podría engañar al analista.

Finalmente, el uso de **modelos comprimidos (Student)** exige precaución. Al reducir la capacidad del modelo, este podría simplificar sus reglas de decisión y apoyarse más en estereotipos superficiales, agravando los sesgos existentes en los datos. Por tanto, es crítico reevaluar la equidad (Recall por clase) específicamente en el modelo destilado antes de su despliegue, como hemos hecho en este estudio.

7. Uso de Gemini

Hemos utilizado Gemini para fines de redacción del informe y maquetación en L^AT_EX.

Referencias

- [Bej20] Matei Bejan. Multi-Lingual Lyrics for Genre Classification. <https://www.kaggle.com/datasets/mateibejan/multilingual-lyrics-for-genre-classification>, 2020.
- [Car22] Carlos G. Genius song lyrics with language information. <https://www.kaggle.com/datasets/carlosgdcj/genius-song-lyrics-with-language-information>, 2022.
- [DdNM25] Carmen Maeso Soler Daniel del Nuevo Montero, Diego Esclarín Fernández. Memoria proyecto 1: Clasificación de géneros musicales (teacher), 2025. Práctica 1 de la asignatura PLN2.
- [Goo24] Google. LaTeX Memoria PLN: Estructura y Plantilla. Gemini 3 Pro, 2024. Accedido el 09-12-2025.
- [Uri25] Nicolás Hernán Rodríguez Uribe. Material docente de la asignatura procesamiento del lenguaje natural 2 (pln2), 2025. Universidad Rey Juan Carlos. Incluye: T2, T3, T7.
- [VSP⁺17] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in neural information processing systems*, 30, 2017.